DEVELOP SMART WATER SOLUTIONS

Guide 02 | Student extensions | Engineering models | Test and improve

Use the working Smart Drops application to develop and test a building-scale water-management idea. Start with a small sewage heat-recovery improvement, extend rainwater collection next, and then tackle a control, efficiency or resilience feature. AI can help write code; your group must explain the assumptions, units and results.

1 EXPLORE	2 DEFINE	3 IMPLEMENT	4 TEST	5 IMPROVE
Try the existing tools	Choose one small goal	Add a clear function	Check units and limits	Use peer feedback

1. Follow a repeatable development workflow

Complete Guide 01 first. Run npm run dev from your local SmartWaterApp folder and create a Git branch for your work. The four tabs are Monitoring, Management, Maintenance and Development. The symbolic building, meters and 2026 water data are educational assumptions, not actual Kremmergården measurements.

- Explore the chosen tool before coding. Read README.md and its model note in docs/. Search for STUDENT EXTENSION in the source and STUDENTS EXTENSIONS in the interface. Identify what works already and what your idea adds.
- Define a small operator task, inputs with units, assumptions, outputs, a comparison case and a few expected results. Sketch the interaction before asking AI to implement it.
- Keep calculations in a model module, controls in its component, styles in its scoped CSS and checks in tests/. Reuse the shared hourly data. Store proposed designs separately from the baseline.
- Ask AI for one bounded change. Review the diff, run npm test and npm run build, and try the tool in the browser. Test normal, zero and failure conditions. Save a Git checkpoint after a working step.
- Ask another student to complete the task without help. Record at least two specific observations, improve the feature and repeat the same task. Explain any trade-off that remains.

FIRST PROMPT: Inspect the existing [tool] and its STUDENT EXTENSION marker. Explain its current inputs, outputs and calculation files. Propose one small improvement and three checks. Do not edit files yet.

2. Start simple with sewage heat recovery

Open Development > Additional Systems > Sewage Heat Recovery, or enable Sanitary drainage and select the exchanger symbol. Choose an exchanger, add heat consumers and set wire allocations to 100%. Move or rename nodes and save a local design. Try water preheating, ventilation, space heating and thermal storage to understand what each allocation means.

Understand the existing calculations

The selected-hour heater gauge estimates electrical input in kW and the sewage gauge shows temperature in degrees C. The two energy gauges sum hourly values from January 1 through the selected simulation hour: baseline heater electricity and recovered thermal heat, both in kWh. kW is power; kWh is energy. Temperature is an instantaneous hourly reading, not a yearly sum.

```
Available thermal kW = sewage L/min / 60 x 4.186 x max(0, sewage deg C - cold deg C)
Heater electric kWh per hour = useful heater thermal kWh / 0.98
Recovered kWh per hour = min(40% of heater electric kWh, available heat x effectiveness,
exchanger limit)
```

The 40% figure is a teaching target, not a measured saving. Available sewage heat and equipment capacity can reduce it; the supplied full-year example is about 39%. Different selections or weather/water-load scenarios can give less. Recovery is zero without a placed exchanger and a connected heat consumer. Only the water-preheating allocation directly reduces the estimated water-heater electricity input. Other heat uses need their own demand models.

Your first development task

Add one editable engineering assumption, such as the 40% recovery target or a fouling factor, and show its effect on the existing year-to-date energy result. Connect it to the existing designer and local save. Keep the window within the viewport and preserve the selected module, baseline meters and other tabs.

- Work in src/heat-recovery-model.js and src/components/heat-recovery-designer.js. Presets are in data/heat_recovery_presets.json; styles and persistence have their own heat-recovery files. Read docs/HEAT-RECOVERY-DESIGNER.md.
- Check zero sewage flow, no positive temperature difference, no exchanger/consumer and a low heater demand. Recovered heat cannot exceed the available heat or the configured limit. Saving and reopening must retain the new assumption.

IMPLEMENTATION PROMPT: Add an editable recovery target to the existing sewage heat-recovery designer and saved design. Reuse heatEnergySummary, cap recovery by available sewage heat, keep kW and kWh distinct, and add zero-flow and year-to-date tests. Preserve unrelated files and baseline CSVs.

Next steps: add hourly consumer heat-demand limits, auxiliary pumping energy, fouling/cleaning, temperature feasibility and thermal storage. Keep sewage and clean-water circuits separate in the conceptual design. Saving this preview does not apply savings to Monitoring automatically; any future link must use a separate scenario.

3. Extend rainwater collection and reuse

Open Development > Additional Systems > Rainwater Collector. The roof frame, connected consumer cards and weather panel already work. Edit roof area, runoff, collection efficiency, storage and pumping capacity. Select irrigation, flush tanks, cooling or parking washing. Inspect the hourly weather and monthly precipitation graphs, then save and reopen the design.

Use Import CSV for a complete hourly 2026 file or Get API data for Open-Meteo historical reanalysis. Earlier complete years are clearly replayed on the 2026 timeline; they are not actual 2026 building weather. API access needs internet; supplied/imported rows and saved designs remain local. Reset returns to the supplied synthetic weather.

```
Collected litres = precipitation mm x roof area m2 x runoff x collection efficiency
Previous storage + inflow = supplied rainwater + overflow + end storage
Mains backup = consumer demand - supplied rainwater
```

Your first rainwater extension can add a configurable first-flush loss or a demand schedule for one consumer. Keep the existing hourly balance and shared dataset. Show how a wet spell and a dry spell change useful supply, overflow and mains backup. Add explicit treatment/quality assumptions for non-potable purposes; the model does not yet simulate snow storage or melt.

- Work in src/rainwater-model.js, src/components/rainwater-designer.js and data/rainwater_presets.json. Weather inputs are in data/rainwater_weather_2026.csv; API code is separate in src/weather-api.js. Read docs/RAINWATER-DESIGNER.md.
- Test zero rainfall, an empty/full tank, zero pump capacity, high demand and missing imported values. Storage must stay within capacity and every hour must conserve water. API failure must retain the previous weather data.

IMPLEMENTATION PROMPT: Extend the existing rainwater model with an editable first-flush loss. Preserve its hourly storage balance, weather import and local saving. Compare wet/dry periods and add checks for zero rain and overflow. Do not replace the existing designer or baseline water dataset.

4. Develop the tagged student extension panels

After the two guided examples, choose the extension assigned by your teacher. The red STUDENTS EXTENSIONS tags mark UI starting points. Management contains Smart Pump Control, Extreme Conditions Management and Water Saving. Development contains Energy Efficiency, Additional Systems and Retrofit / Proposals. The designer source also includes searchable STUDENT EXTENSION comments.

Smart Pump Control

Start with a transparent pressure or operating-schedule rule. Inputs include hourly demand, required head, efficiency and capacity. Compare it with the supplied pump-energy model in src/pump-model.js. Report energy in kWh, peak kW and unmet demand/pressure. Changing a control label alone cannot create savings; the simulated operating conditions must change. More advanced studies may compare regression, neural networks, predictive control or reinforcement learning after validating the simple rule.

Water Saving and Energy Efficiency

For Water Saving, model one fixture, leakage-reduction or demand-schedule intervention against a reference demand. For Energy Efficiency, compare pressure optimization, hot-water setpoints or the heat-recovery preview with comfort and operating limits. Report assumptions, water/energy saved and any service shortfall. Reuse src/data-model.js, src/scenarios.js and the pump/heat models rather than multiplying independent meters by unrelated random factors.

Extreme Conditions Management and Maintenance

Simulate two cases such as pump failure and peak demand, or a supply outage and intense rain. Use event severity, duration, supply capacity and storage; show affected loads, unmet demand and a reasoned response. Maintenance already includes leak investigations and pump-energy degradation examples. Extend detection or uncertainty in an isolated model; an alarm should explain the threshold crossed. All controls remain simulated.

5. Use retrofit scenarios to challenge your solution

In Development > Retrofit / Proposals > Open Building Editor, select a whole module, choose a new tenant type, adjust area/occupancy/schedule or import hourly cold/hot loads. Apply a local scenario and inspect its

effects on floors, meters, demand and pump energy. The supplied baseline remains separate. Restore supplied baseline in the editor to compare with the starter model.

Use the changed demand to test your extension. Does pump control serve a new restaurant? Is rainwater storage sufficient for its flushing demand? Does wastewater heat availability change? Use the existing retrofit engine as a scenario tool; extending it is optional unless assigned. Its files and CSV rules are described in docs/DATA-SCHEMA.md.

6. Validate and hand in a reproducible result

- Provide the working source and a short README explaining how to run the feature. Include changed source/presets, dependencies and relevant inputs; exclude node_modules/ and generated dist/. Browser-only designs need an export or a documented set of inputs.
- Explain the operator problem, inputs, units, assumptions and calculation/control rule. Distinguish thermal energy, electrical energy, flow, volume and temperature. State which results are simulated.
- Compare at least two meaningful scenarios and include at least three checks: a normal case, a limiting/zero case and a failure or missing-data case. Record npm test and npm run build outcomes and confirm other tabs, meters and local saving still work.
- Include two peer-testing observations, the changes you made and evidence of improvement. Submit through the teacher's assigned branch/repository/folder route, not the shared starter main branch.

REVIEW PROMPT: Review only my student extension. Check units, hourly/annual balances, zero and failure inputs, local persistence and baseline isolation. Show any defect with a small reproducible example. Explain the engineering result so our group can defend it.